

Wallet & P2P Transfer

Design and reasoning | Python, FastAPI, PostgreSQL | September 13, 2026

Data model. `users` stores identities and SHA-256 hashes of high-entropy bearer tokens. `wallets` has one unique owner and a checked BIGINT paise balance. `transfers` stores sender, recipient, integer amount, final decision, timestamp, and a unique (`user_id`, `idempotency_key`). Successful transfer records are an immutable movement history; balances can be reconciled from opening funds and successful movements. This exercise does not implement a separate double-entry accounting ledger.

Simplest correct transaction. At PostgreSQL Read Committed, reserve a transfer key with a unique insert, lock both wallets individually in ascending UUID order using `FOR NO KEY UPDATE`, check funds and recipient capacity, update both balances, record the decision, and commit. A decline commits its result without moving money. No network calls occur inside the transaction. The temporary pending state is uncommitted and never intentionally returned or committed. Balance constraints are a final guard. Fixed lock order handles simultaneous A-to-B and B-to-A transfers; the chosen lock mode also avoids upgrading foreign-key key-share locks to incompatible `FOR UPDATE` locks. Merely using a conditional debit would not prevent a two-wallet deadlock.

Idempotency and failure. The unique key and movement commit together. A losing insert waits, then a separate `SELECT` reads the committed winner. Compare validated wallet IDs and integer amount; a mismatch returns 409. Replays include original declines. Before-commit failure rolls everything back; after-commit response loss is resolved by retrying the same key. Keys do not expire. Database errors and bounded lock waits return 503; clients retain their key because commit outcome can be uncertain.

Alternatives and tradeoffs. Serializable isolation is valid but adds transaction retry handling. Redis locks add another failure boundary, and process locks do not coordinate replicas. A queue/outbox is unnecessary for a transaction entirely inside one database. We prioritize consistent money movement over accepting writes during database outages. Hot wallets serialize and bound throughput; unrelated wallet pairs can proceed concurrently. This is one committed effect per key, not guaranteed network delivery or unlimited availability.

Operate and verify. A non-root multi-stage image runs with PostgreSQL in Compose. Real-database tests cover races, two app instances, rollback after debit, overflow, authorization, restart replay, and lock timeouts. HTTP probes reconcile balances with responses. JSON logs carry request correlation IDs and emit committed outcomes; Prometheus histograms/counters expose latency and domain events. Process metrics and stdout logs are operational evidence, not a durable ledger: a crash just after commit can miss an event, and counters reset on restart.

AI directed versus decided. The contributor selected Python, requested the wallet exercise, reviewed the proposed approach, and accepted the recommendations. The AI assistant proposed FastAPI/Pydantic, ordered locks, per-user keys, demo and hosting defaults, then implemented code, tests, packaging, and documentation. Acceptance of those recommendations does not mean they were independently authored by the contributor. Repository history reflects actual implementation checkpoints.

Cost and capacity. Target: INR 0 using Render Free and Neon Free, within provider limits. Cold starts and capped database connections affect latency; no production SLA or unmeasured throughput claim is made. Actual deployment and test evidence are recorded separately; passing local tests alone does not establish deployed correctness.